Ember — a full-stack AI chatbot

A complete, working AI chatbot: Node/Express backend, vanilla JS frontend, persistent chat history (SQLite), and real AI replies from a **free** LLM API (no paid key needed).

```
chatbot/
├── server.js           # Express backend + SQLite + LLM calls
├── package.json
├── .env.example         # copy to .env and add your free API key
├── render.yaml           # one-click deploy config for Render.com
└── public/
    ├── index.html       # chat UI
    ├── style.css
    └── app.js
```

Why Groq, and is it really free?

To actually *talk back*, a chatbot has to call some LLM. There's no production-quality LLM API that needs literally zero signup — but **Groq** (https://groq.com) gives free API keys with no credit card, a generous free rate limit, and very fast open-source models (Llama 3.3 70B). That's what this app uses. If you'd rather use Claude or OpenAI instead, see "Swapping the model" below — it's a ~10 line change in `server.js`.

1. Run it locally

Requirements: Node.js 18+ installed on your computer.

```
cd chatbot
npm install
cp .env.example .env
```

Then get a free key at **https://console.groq.com/keys** (sign up, click "Create API Key") and paste it into `.env`:

```
GROQ_API_KEY=gsk_your_actual_key_here
```

Start the server:

```
npm start
```

Open **http://localhost:3000** — you now have a working chatbot with persistent history stored in `chat.db` (SQLite, created automatically).

2. Deploy it live (free)

Option A — Render.com (recommended, easiest)

1. Push this folder to a new GitHub repository.
2. Go to https://render.com → sign up (free) → **New +** → **Blueprint**.
3. Connect your GitHub repo. Render will detect `render.yaml` automatically.
4. When prompted, paste your `GROQ_API_KEY` as the environment variable.
5. Click **Apply** / **Deploy**. In ~1-2 minutes you'll get a public URL like `https://ember-chat.onrender.com` — that's your chatbot's live link.

> **Note on the free tier:** Render's free web services spin down after ~15 minutes of inactivity (the first request after that takes ~30s to wake up), and the local SQLite file is wiped on redeploy/restart. For a personal project or demo this is completely fine. For permanent history, swap in a hosted free database like Turso or Supabase Postgres later — the `stmts` object in `server.js` is the only place you'd need to touch.

Option B — Railway.app

1. Push to GitHub, go to https://railway.app → **New Project** → **Deploy from GitHub repo**.
2. Add the `GROQ_API_KEY` environment variable in the Railway dashboard.
3. Railway auto-detects Node and runs `npm start`. You'll get a public `*.up.railway.app` URL.

Option C — Deploy without GitHub (Render CLI / manual upload)

If you don't want to use GitHub, you can zip this folder and use Render's "manual deploy" upload option, or run it on any VPS (e.g. a $0 Oracle Cloud free-tier instance) with `pm2` or a systemd service to keep it alive.

Features included

- **Real conversation** with an actual LLM (Groq's Llama 3.3 70B by default)

- **Persistent chat history** — every session and message stored in SQLite
- **Multiple conversations** — sidebar to create, switch between, rename (auto-titled from your first message), and delete chats
- **Markdown-lite rendering** — code blocks, inline code, bold text
- **Typing indicator**, auto-resizing input, mobile-responsive layout
- **Health check endpoint** (`/api/health`) for uptime monitoring

Swapping the model (optional)

To use Anthropic's Claude or OpenAI instead of Groq, only `callGroq()` in `server.js` needs to change — swap the URL, headers, and body shape to match that provider's `/v1/messages` (Anthropic) or `/v1/chat/completions` (OpenAI) endpoint, and add the equivalent API key to `.env` .

Troubleshooting

- **"No GROQ_API_KEY set"** — you haven't created `.env` or forgot to paste the key in (locally), or forgot to add the env var in your hosting dashboard (live).
- **Chat history disappears after redeploying on Render free tier** — this is expected; the free tier's disk isn't persistent across deploys. Use a hosted database for permanent storage (see note above).
- **Slow first response after inactivity** — normal "cold start" behavior of free hosting tiers waking the server back up.